

Introduction Pynimation

The goal of this lab is to set up the environment for the animation part of the CG course, which uses a tool developed in python. We will first go through the installation steps, then go through a number of examples for you to become familiar with the framework.

1. Installation

First, you will be asked to download the zip file containing the code to use in this project then to unzip the file to the location where you will be working from in this project

- <https://drive.google.com/file/d/1O6ZXowJrJzBCJXLgZ9IHwQv4aemkbQGX>

The information for setting up pynimation is available in `docs/index.html`, under the Development/Introduction section, but here are the main steps as a reminder

- Install dependencies: start a terminal and run the command
 `pip install --upgrade -r requirements.txt`
 (if you have problems with your python version, frozen requirements_3xx.txt files are available for specific 3.xx python versions in the folder requirements_backups)
- Add Pynimation to Python path, so that its location is known when you are running your program

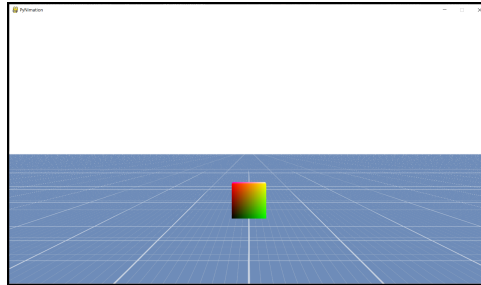
This will give you access to all the functionalities of the pynimation tool (loading animations and 3D objects, accessing and modifying object positions, including cameras and character joint locations) that will be used in the project.

For information, the structure of the code that is provided is roughly the following:

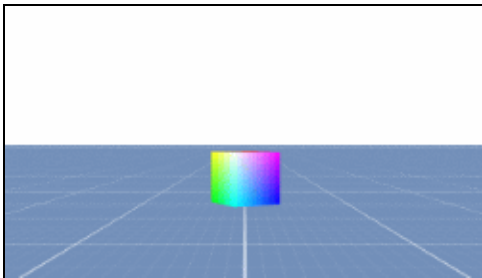
- docs: documentation of pynimation
- examples: a number of examples about loading and displaying objects, animations, etc.
- **labs: source code to be filled for the project**
- pynimation: source code of the core of the pynimation tool
 - anim: code to handle character animations
 - common: default structures
 - data: animations, 3D models, shaders and textures to display the 3D scenes provided for the project
 - io: animation importers and exporters
 - similarity: structures to compute similarity metrics (Including metrics to be completed by you)
 - viewer: various OpenGL, camera, light, and other displayable elements

2. Getting used to Pynimation - Transforms

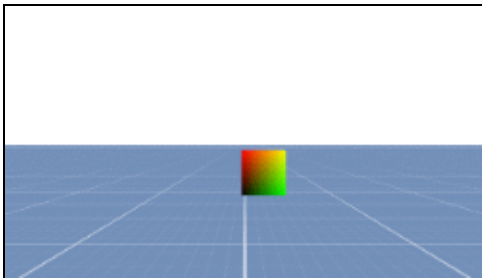
Now, as a first step, we will get familiar with using Pynimation to load and move objects, as well as to load and display animations, which will be the main elements to be used in the following of the project. To become familiar with the project, we first ask you to run the examples/lab_01.py, which should display the following window



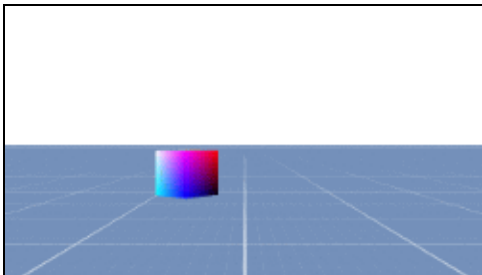
The first step of the lab is to then become familiar with how transforms work. We therefore ask you to perform three tasks.



Make the cube rotate on itself around the Y axis



Make the cube rotate clockwise on the Y axis around a fixed point (at 1m distance from this point) while the cube itself does not rotate around its own Y axis



Make the cube rotate clockwise on the Y axis around a fixed point (at 1m distance from this point) while the cube itself rotates anti-clockwise its own Y axis

To do so, you can for instance rely on the following method. More information are available through the documentation of the project

Access to the current time (time in the application is handled by the class Player)

```
self.viewer.player.time
```

Creates a rotation of degValue degrees around the Y (vertical) axis. Generally, transforms and rotation creations are handled by the class Transform (see the documentation)

```
rot = Transform.rotationMatrixYDeg( degValue )
```

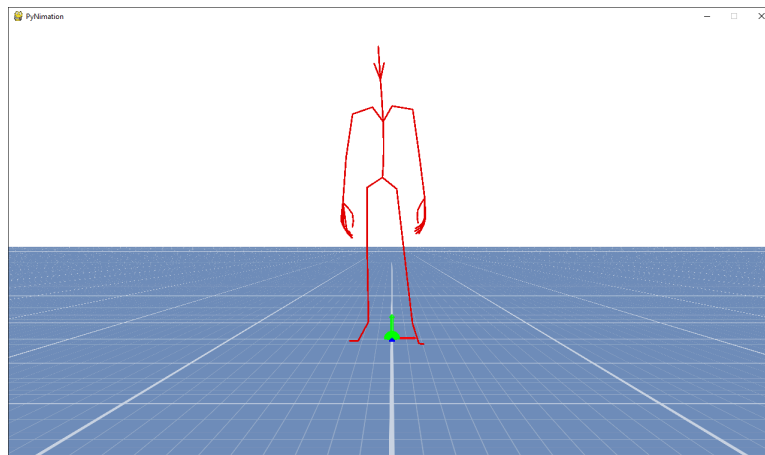
Assign the rotation of the global transformation of the object cube

```
self.cube.globalTransform.setRotation(rot.getRotation())
```

3. Getting used to Pynimation - Animations

In this second example, you will be getting used to load and display animations on a frame-by-frame manner.

Please now open then run the examples/lab_02.py, which should display the following window



In this example, the class MyCharacter() handles the loading of the animation and the visual representation (StickFigure in that case). For the purpose of the example, the animation frame is displayed manually in the beforeDisplay() function

```
currentTime = self.viewer.player.time  
frame = self.fbx_character.animation.getFrameFromTime(currentTime)  
self.fbx_character.mesh.animate(frame)
```

To become familiar with the animation basics, your goal is to specify the position of the sphereR and sphereL objects at each frame so that they follow respectively the left and right ankles. In particular, you can rely on

Getting/Setting the global transformation of the id joint through

```
frame.globalTransforms[id]
```

You can also access the id of a specific joint using

```
frame.skeleton.leftAnkle.id
```

You can also test whether it is working as it should using other motion capture files, such as ybot@WalkingForwardLoop.fbx